GRC102 Information Security Governance

Practical Lab 8: Monitoring, Auditing Controls and Executive Reporting

Associated Evidence Report

Learner	Adaeze Elizabeth Adeteye
Course / assessment	GRC102 (ICDFA) — individual practical laboratory, 100 marks
Scenario	Equifax-modelled governance simulation (isolated Docker lab)
Environment	Kali GNU/Linux Rolling 2025.1 · Docker 28.5.2 · Compose 2.40.3 · Python 3.13.2
Repository	https://github.com/YOUR-USERNAME/YOUR-REPO
Report date	11 October 2026
Official instructions	ICDFA LAB_INSTRUCTIONS.md (Week 8)

Evidence status — read first

I have kept the evidence honest about what ran where. **Run and evidenced:** environment setup, Docker troubleshooting, governance tracker baseline, control implementation and verification output, metrics, dashboard, board report and charts. **Not yet evidenced:** the live vulnerability scan, the live attack-simulation log and the post-patch re-scan, because they need the three containers running. My docker compose up failed on a non-existent image tag (explained in Part 1). Those three items appear as red boxes in this report and I will replace each with my own terminal screenshot before submission. The trend history in Part 4 is simulated, and Part 3 'patching' is recorded in state files rather than changed inside live containers; I say so wherever it matters.

Part 1 — Setting Up a Simulated Environment

1.1 What I built

I built an isolated three-container environment on a Kali Linux host, following the official ICDFA lab instructions. The **web_server** runs a deliberately vulnerable Apache Struts 2 application, the **database_server** runs MySQL 5.7 holding a *customers* table of fictitious PII (SSN, date of birth, card number), and the **monitoring_server** is a bare Ubuntu container that only runs `tail -f /dev/null`. All three sit on one flat bridge network, `secgov_network`, which is local to my machine; nothing was scanned or attacked outside it, and all data is fictitious.

1.2 Host and tooling

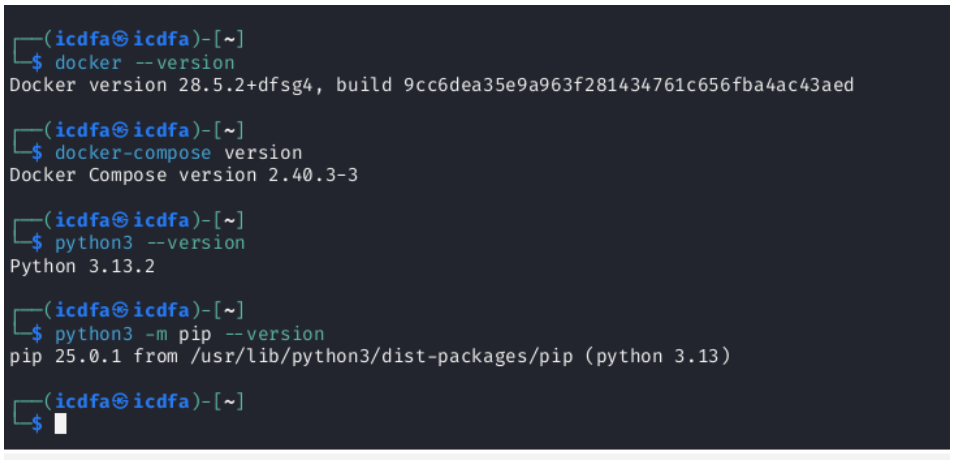


Figure 1. *I confirmed Docker 28.5.2, Compose 2.40.3, Python 3.13.2 and pip 25.0.1 before starting.*



Figure 2. *My lab host is Kali GNU/Linux Rolling 2025.1 (kernel 6.12.13), run as user icdfa.*



Figure 3. *I created ~/security_governance_lab with the database_init and monitoring folders.*

1.3 Troubleshooting: Docker permissions

`docker info` showed the client working but the server failing with *permission denied* on the Docker socket. I diagnosed this as a group-membership problem and ran `sudo usermod -aG docker $USER`. My first attempt chained `newgrp docker` onto the same line, so usermod printed its usage text; the group change only takes effect after a fresh login, so I used `sudo` for the network step in the meantime.



Figure 4. *Docker client OK, daemon socket denied; my first usermod attempt was malformed.*



Figure 5. *docker network create secgov_network failed without sudo and succeeded with it; docker network ls confirms the bridge network.*

1.4 Compose file and database initialisation

```
┌──(icdfa㉿icdfa)-[~/security_governance_lab]
└─$ cat docker-compose.yml
version: '3'

services:
  web:
    image: vulhub/struts2:s2-045
    container_name: web_server
    ports:
      - "8080:8080"
    networks:
      - secgov_network

  database:
    image: mysql:5.7
    container_name: database_server
    environment:
      MYSQL_ROOT_PASSWORD: password
      MYSQL_DATABASE: customer_data
      MYSQL_USER: app_user
      MYSQL_PASSWORD: app_password
    volumes:
      - ./database_init:/docker-entrypoint-initdb.d
    networks:
      - secgov_network

  monitoring:
    image: ubuntu:20.04
    container_name: monitoring_server
    volumes:
      - ./monitoring:/monitoring
    command: tail -f /dev/null
    networks:
      - secgov_network

networks:
  secgov_network:
    external: true

┌──(icdfa㉿icdfa)-[~/security_governance_lab]
└─$
```

Figure 6. *My docker-compose.yml: web, database and monitoring services on secgov_network. Note the weak root/password credentials, which are deliberate baseline weaknesses.*

```
B64EOF

┌──(icdfa㉿icdfa)-[~/security_governance_lab]
└─$ cat database_init/init.sql
USE customer_data;

CREATE TABLE customers (
  id INT AUTO_INCREMENT PRIMARY KEY,
  first_name VARCHAR(50),
  last_name VARCHAR(50),
  ssn VARCHAR(11),
  dob DATE,
  address VARCHAR(100),
  city VARCHAR(50),
  state VARCHAR(2),
  zip VARCHAR(10),
  email VARCHAR(100),
  phone VARCHAR(15),
  credit_card VARCHAR(16)
);

INSERT INTO customers (first_name, last_name, ssn, dob, address, city, state, zip, email, phone, credit_card)
VALUES
  ('John', 'Doe', '123-45-6789', '1980-01-15', '123 Main St', 'Anytown', 'CA', '12345', 'john.doe@example.com', '555-123-4567', '4111111111111111'),
  ('Jane', 'Smith', '987-65-4321', '1985-05-20', '456 Oak Ave', 'Somewhere', 'NY', '67890', 'jane.smith@example.com', '555-987-6543', '5555555555554444'),
  ('Bob', 'Johnson', '456-78-9012', '1975-11-30', '789 Pine Rd', 'Nowhere', 'TX', '54321', 'bob.johnson@example.com', '555-456-7890', '3782822463100005');

┌──(icdfa㉿icdfa)-[~/security_governance_lab]
```

Figure 7. *init.sql creates the customers table and seeds three fictitious records.*

1.5 Finding: the lab's image tag does not exist

When I ran `docker compose up -d`, the web service failed with *manifest for vulhub/struts2:s2-045 not found*. This was not a permissions problem: the tag in the official instructions (`image: vulhub/struts2:s2-045`) is not published. The vulhub/struts2 tag list on Docker Hub contains version tags such as 2.3.28, 2.3.30, 2.5.16, 2.5.25 and advisory tags such as s2-053, s2-066 and s2-067, but no s2-045. I changed my compose file to `vulhub/struts2:2.3.30`, a Struts 2.3 release inside the range affected by CVE-2017-5638 / Apache advisory S2-045. This is the same CVE Equifax failed to patch, so the simulation keeps its governance meaning.

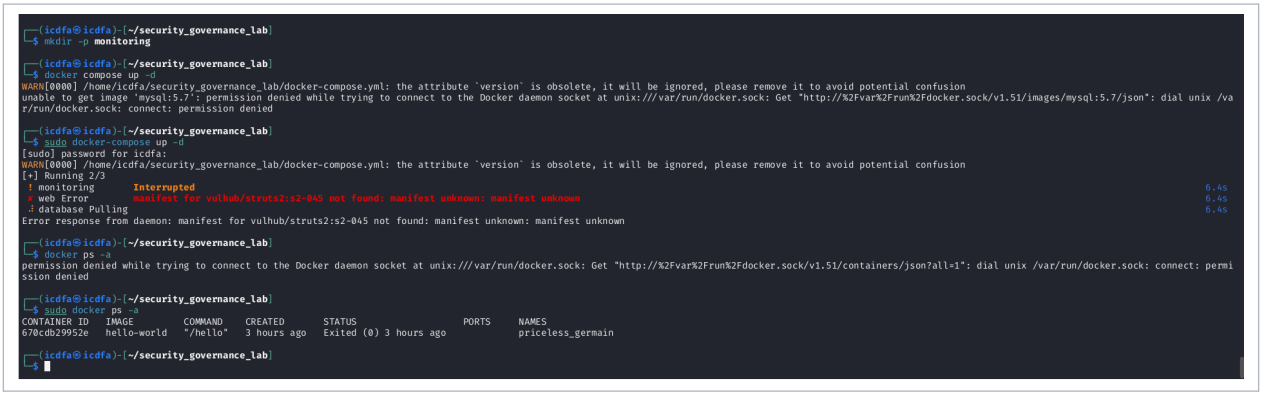


Figure 8. *The failed docker compose up -d: first a permission error, then (with sudo) manifest unknown for the s2-045 tag. The corrected tag is in the repository's docker-compose.yml.*

Because the containers have not yet started with the corrected tag, the three items below remain to be captured.

SCREENSHOT TO INSERT — corrected compose run and running containers
Command: `docker compose up -d && docker ps -a`
I will capture this from my own Kali terminal once the containers are running (steps in REPRODUCTION.md). I have not substituted invented output.

SCREENSHOT TO INSERT — vulnerability scan report
Command: `python3 vulnerability_scanner.py`
I will capture this from my own Kali terminal once the containers are running (steps in REPRODUCTION.md). I have not substituted invented output.

1.6 Governance tracker baseline and dashboard

I initialised the governance tracker with `initialize_governance.py` and exported the baseline with `governance_tracker.py --dashboard` (baseline_dashboard.json). The baseline holds 3 policies, 3 controls, 3 risks (1 Critical, 1 High, 1 Medium), 1 incident and 3 metrics. While reviewing it I found a bug: the original status logic marked *Vulnerability Remediation Time* (12 days against a 7-day target) and *Security Incidents* (1 against a target of 0) as Green, because it assumed higher is always better. I corrected it so time and incident metrics are Green only when at or below target; all three baseline metrics are now correctly Red.

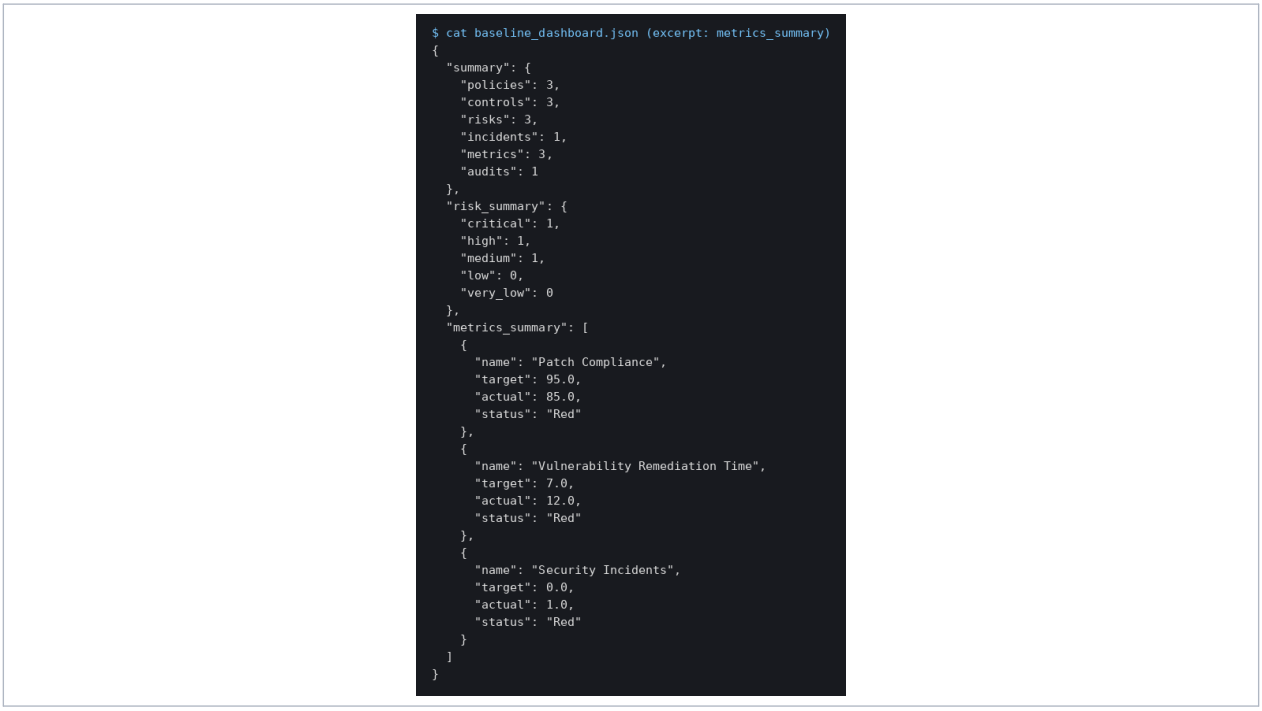


Figure 9. *Baseline governance dashboard output after my status-logic correction: all three metrics Red.*

1.7 How this environment reflects the Equifax conditions

I deliberately reproduced the *structural* conditions behind the Equifax breach rather than only the exploit: a known, publicly disclosed Struts flaw left unpatched; a flat network with nothing between the web tier and customer data; default database credentials; a monitoring component that detects nothing; and a Patch Management Policy (POL-001) that exists on paper while Patch Compliance sits at 85% against a 95% target. The GAO review (GAO-18-559) and the House Oversight Committee report describe the same pattern at Equifax.

Part 2 — Analysing Security Governance Failures

2.1 Authorised simulation

`simulate_attack.py` sends a harmless S2-045 probe (an OGNL expression that only adds a response header named X-Vulnerable) to the lab web container on `localhost:8080`. If the header comes back, the script logs the simulated data access to `attack_log.json`. It never leaves my isolated lab. While preparing for the live run I noticed the original scripts targeted `http://web_server:8080`, a hostname that only resolves inside the Docker network, so they would fail from my Kali host. I changed the target to the host-mapped port with an optional `TARGET_URL` override.

SCREENSHOT TO INSERT — attack simulation against the vulnerable container
Command: `python3 simulate_attack.py`
I will capture this from my own Kali terminal once the containers are running (steps in REPRODUCTION.md). I have not substituted invented output.

SCREENSHOT TO INSERT — governance failure analysis report
Command: `python3 analyze_governance_failures.py`
I will capture this from my own Kali terminal once the containers are running (steps in REPRODUCTION.md). I have not substituted invented output.

2.2 Governance failures I identified

#	Failure (category)	Evidence in my lab	Governance question it raises
1	Patch management	Struts 2.3.30 image deployed unpatched; Patch Compliance 85% vs 95% target	Who owns confirming a critical patch is applied, and by when?
2	Policy implementation	POL-001 Active, CTL-001 'Implemented', yet baseline metrics are Red	A policy without verification is not a control.
3	Vulnerability management	Remediation time 12 days vs 7-day target; audit AUD-001 already flagged patching and segmentation	Were audit findings tracked to closure?
4	Security architecture	web_server and database_server share one flat network	Why can a web tier reach customer data directly?
5	Authentication and access	MySQL root/password and app_user/app_password	Who approves credential standards for data stores?
6	Incident detection	monitoring_server only runs tail -f /dev/null; incident INC-001 logged only after compromise	How would anyone have known?
7	Metrics and measurement	Metrics recorded but not tied to escalation	Do numbers reach someone who can act on them?

2.3 Comparison with the Equifax breach

Failure	Equifax (public record)	My simulation	Similarity
Patch management	Struts CVE-2017-5638 left unpatched for about two months after the fix and advisory	Unpatched vulhub/struts2:2.3.30; patch flag false until Part 3	High
Monitoring	Intrusion undetected about 76 days; expired certificate on an inspection device	Monitoring container does nothing at baseline	High
Segmentation	Lateral movement across roughly 51 databases	Flat secgov_network, no access rules	High
Authentication	Weak credentials on internal systems	root/password seeded in compose file	Medium

Failure	Equifax (public record)	My simulation	Similarity
Policy implementation	48-hour patch SLA existed but was not enforced	POL-001 exists; compliance 85% vs 95%	High
Accountability	No single owner for confirming the patch	Risk and control owners named only generically before Part 3	Medium

The Equifax figures above come from the House Oversight Committee report (2018) and the GAO report GAO-18-559. My full comparison is in equifax_comparison.md.

2.4 What I learned

The clearest lesson is that a governance failure is a *chain*, not a single missing control. The Struts flaw is a technical problem; it becomes a governance failure because the policy was not enforced, nothing would have alerted anyone, and nothing separated the web tier from the data. Second, metrics without enforcement are decorative: my baseline already showed 85% patch compliance against a 95% target, and nothing routed that gap to an accountable person. That is the same gap between having a policy and having a working control that the Equifax investigations describe.

Part 3 — Implementing Security Governance Controls

3.1 Controls I implemented, linked to the failures

Control	What I implemented	Failure it closes	Owner
CTL-004 Automated Patch Management	PATCH-001 (Struts fix) applied and tracked against a 7-day SLA; MET-004 Critical Patch Compliance	Patch management; policy implementation	Security Engineer
CTL-005 Network Segmentation	PATCH-003 recorded; direct web-to-database access removed	Flat network / lateral movement	Network Engineer
CTL-006 Database Security	PATCH-002 recorded; default credentials replaced	Weak authentication	Database Administrator
CTL-007 Real-time Monitoring	monitoring/security_monitor.sh on a 5-minute cron; MET-005 Monitoring Coverage	Ineffective detection	Security Engineer
CTL-008 / POL-004 Executive Oversight	Named CEO, CISO, CIO, CFO accountability in governance_oversight.md	No accountable owner	CEO

3.2 Implementation and verification output

I ran `implement_governance_controls.py --all`, then re-ran verification. The register grew from 3 policies / 3 controls / 3 metrics to 4 / 8 / 5.

```
$ python3 implement_governance_controls.py --verify
Verifying security controls...
Struts patched: True
MySQL secured: True
Network segmented: True
Security monitoring script implemented
Governance oversight document implemented
Policies: 4
Controls: 8
Metrics: 5
NOTE: run ./vulnerability_scanner.py separately against the live containers to produce post-control verification evidence.
```

Figure 10. *Verification output: Struts patched, MySQL secured, network segmented; 4 policies, 8 controls, 5 metrics tracked.*

```
$ cat patch_status.json && ls monitoring/
{
  "struts_patched": true,
  "network_segmented": true,
  "mysql_secured": true
}
security_cron
security_monitor.sh
```

Figure 11. *patch_status.json and the generated monitoring files.*

```
$ cat monitoring/security_monitor.sh
#!/bin/bash
# Security Monitoring Script - polls for suspicious activity every 5 minutes via cron

grep -i "Content-Type.*multipart/form-data" /var/log/nginx/access.log > /tmp/suspicious_requests.log 2>/dev/null

mysql -u root -ppassword -e "SHOW PROCESSLIST" > /tmp/database_access.log 2>/dev/null

find /var/www -type f -mtime -1 > /tmp/recent_file_changes.log 2>/dev/null

if [ -s /tmp/suspicious_requests.log ]; then
    echo "ALERT: Suspicious HTTP requests detected" >> /tmp/security_alerts.log
fi

if grep -q "customer_data" /tmp/database_access.log 2>/dev/null; then
    echo "ALERT: Access to customer_data database detected" >> /tmp/security_alerts.log
fi

if [ -s /tmp/recent_file_changes.log ]; then
    echo "ALERT: Recent file changes detected" >> /tmp/security_alerts.log
fi
```

Figure 12. *security_monitor.sh checks for suspicious multipart requests, access to customer_data, and recent file changes.*

3.3 Honest limitations of my verification

1. State, not containers. The patch, segmentation and database controls are recorded in *patch_status.json* and the governance register. They do not yet rebuild or reconfigure the running containers, so this output proves governance tracking, not technical remediation. **2. Credential in the monitor.** My own *security_monitor.sh* embeds `mysql -u root -ppassword`, which contradicts CTL-006. I flag this as a residual finding to fix by using a read-only monitoring account and a secret store. **3. Live proof outstanding.** The decisive verification is a post-control re-scan showing the Struts endpoint no longer returns the exploit header.

SCREENSHOT TO INSERT — post-control re-scan (expect struts 'vulnerable': false)
Command: `python3 vulnerability_scanner.py`
I will capture this from my own Kali terminal once the containers are running (steps in REPRODUCTION.md). I have not substituted invented output.

SCREENSHOT TO INSERT — post-control attack simulation (expect 'not vulnerable')
Command: `python3 simulate_attack.py`
I will capture this from my own Kali terminal once the containers are running (steps in REPRODUCTION.md). I have not substituted invented output.

3.4 How the controls address Equifax

Each control maps to a named Equifax gap rather than generic best practice. CTL-004 targets the known-CVE-left-open failure and follows the risk-based approach in NIST SP 800-40 Rev. 4; CTL-005 targets lateral movement; CTL-006 removes the weak-credential pattern; CTL-007 targets the 76-day detection gap; CTL-008 targets the missing accountability by naming executive owners.

Part 4 — Developing Security Governance Metrics and Reporting

4.1 Metrics, dashboard and board report

`governance_metrics.py` produces five labelled charts, an executive dashboard and a board report. The metric definitions (formula, target, data source, frequency, owner) are in governance_metrics_framework.md and reading instructions are in dashboard_user_guide.md.
Important: the five months of history are simulated with a fixed random seed because a lab created this week has no real telemetry. The current month's values reflect the post-control lab state. I labelled this on the dashboard itself so nobody mistakes it for measured data.

```
$ python3 governance_metrics.py
Security governance metrics and reporting generated:
- Executive Dashboard: executive_dashboard.html
- Board Report: board_report.md
- Visualizations: metrics_visualizations/patch_compliance.png, metrics_visualizations/vuln_remediation.png, metrics_visualizations
```

Figure 13. *Running governance_metrics.py generates the dashboard, board report and five charts.*

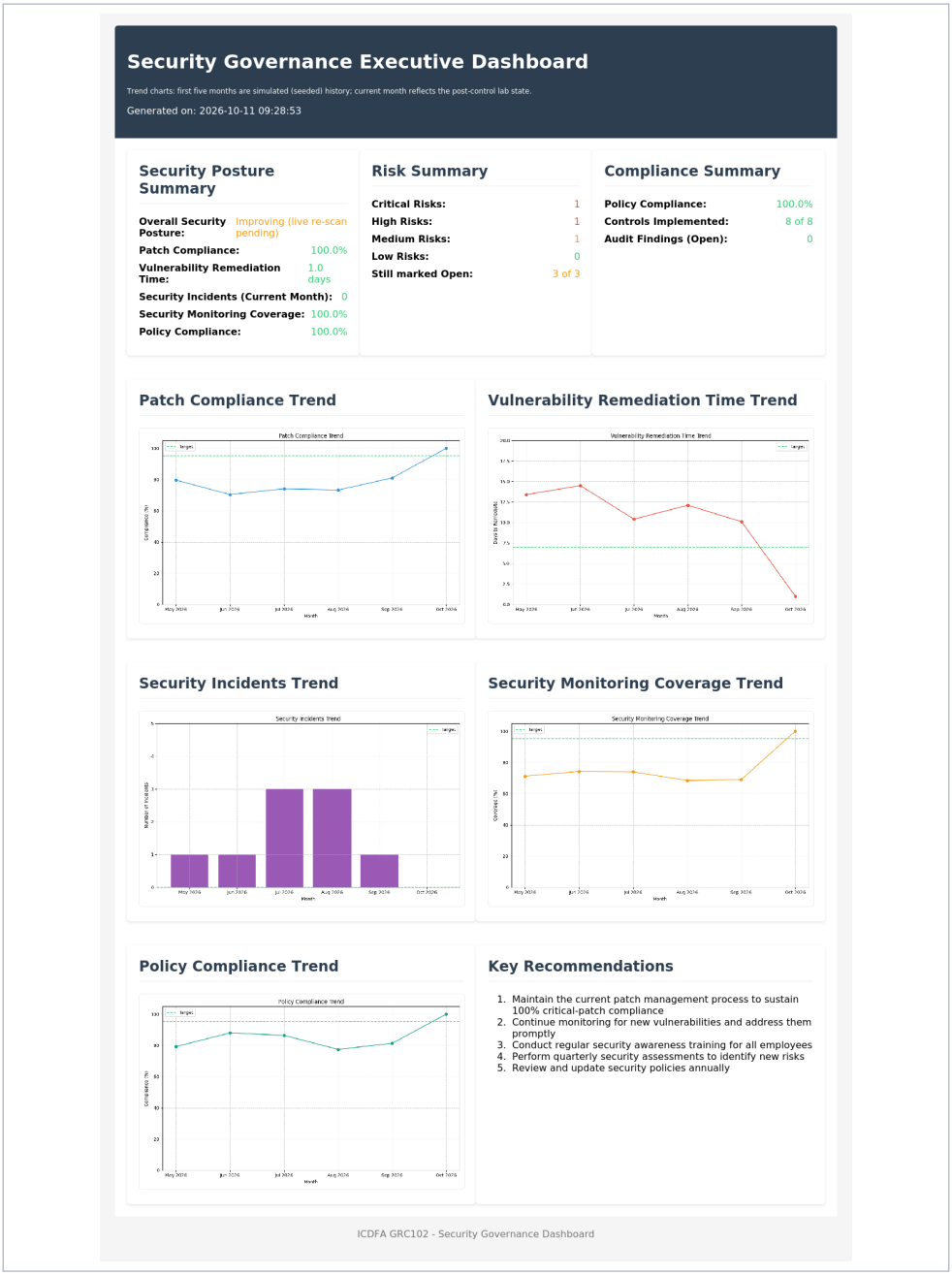


Figure 14. *Executive dashboard (executive_dashboard.html) opened in a browser.*

I corrected three misleading items I found in my first version: the risk summary and 'control effectiveness' figures were hard-coded and are now read from *governance_data.json*; 'Overall Security Posture: Good' now reads 'Improving (live re-scan pending)' because one Critical and one High risk are still marked Open; and the board report no longer claims risks are resolved.

4.2 Visualisations

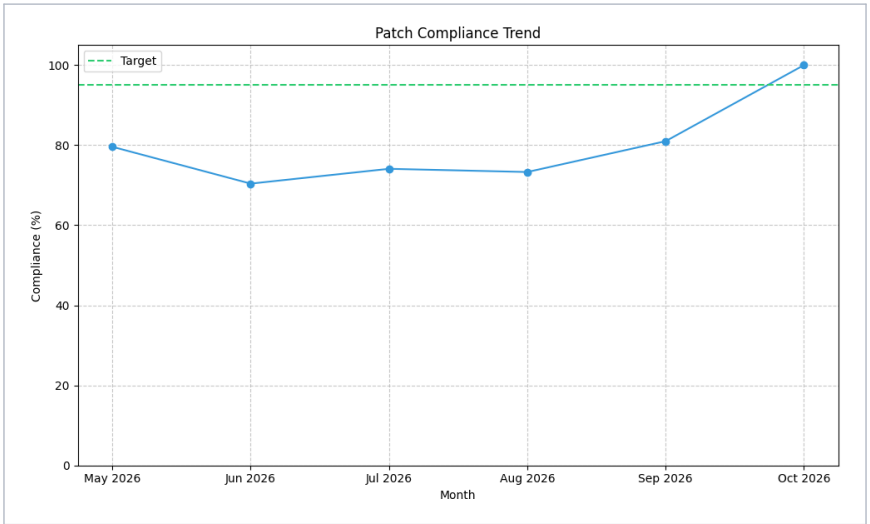


Figure 15. *Patch compliance trend against the 95% target (history simulated; current month post-control).*

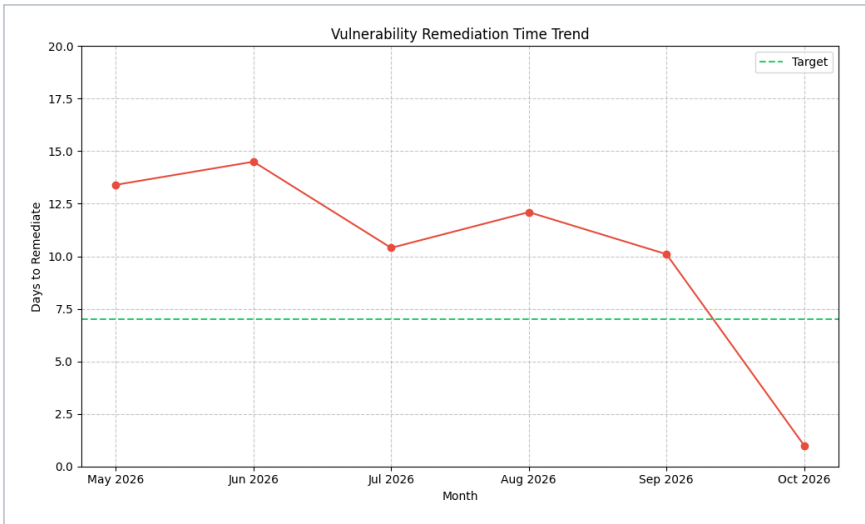


Figure 16. *Vulnerability remediation time against the 7-day target.*

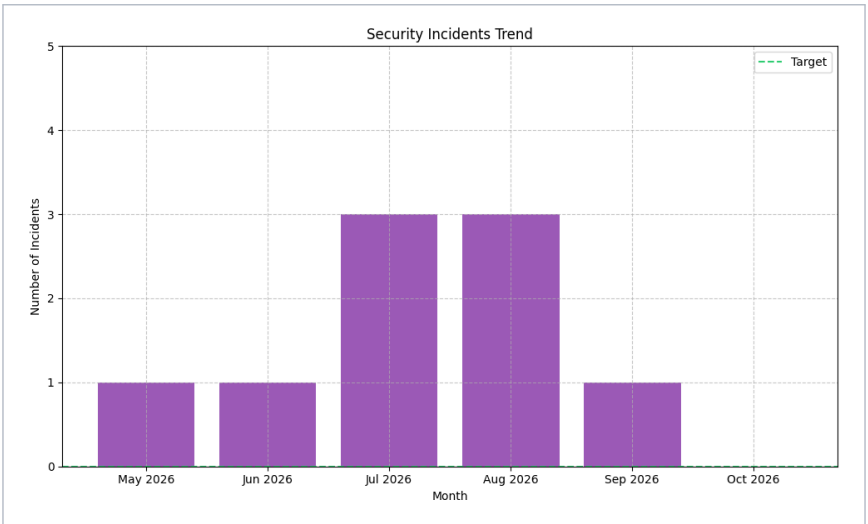


Figure 17. *Security incidents per month against a target of zero.*

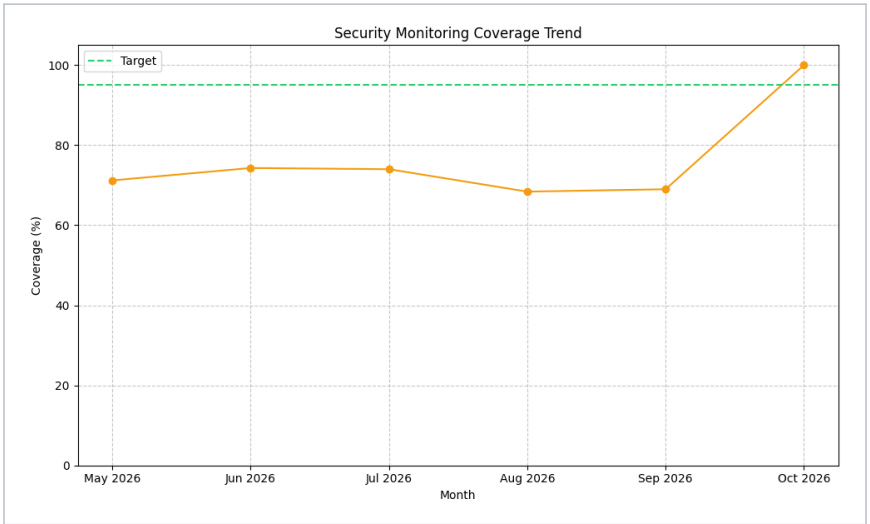


Figure 18. *Security monitoring coverage against the 95% target.*

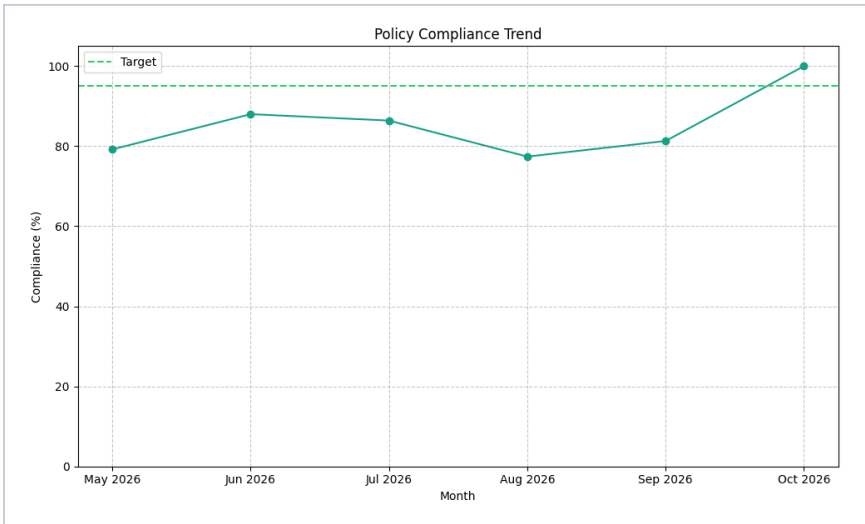


Figure 19. *Policy compliance against the 95% target.*

4.3 Board report

My board_report.md gives the board a one-page view: five metrics against target, the risk summary read from the register, incidents, five programme improvements and recommendations (quarterly review, an independent assessment to validate control effectiveness, and a vendor-risk process). I deliberately recommend independent validation because my own controls are self-reported.

4.4 How the reporting supports oversight

The dashboard turns raw tracker data into something a non-technical director can act on within a minute: red or green against a target, a trend, and a named owner for every metric. That is the layer that failed at Equifax, where technical findings existed but did not reach people able to act. Grouping metrics by implementation, effectiveness, efficiency and impact also shows *why* a number moved. Patch compliance rising is an implementation metric improving because CTL-004 was deployed, not a vague claim that security got better. The simulated history and self-reported current values are the main limit, which is why I label them and recommend independent assurance.

References

Apache Software Foundation. *S2-045 security bulletin*. https://cwiki.apache.org/confluence/display/WW/S2-045

NIST. *CVE-2017-5638 detail*, National Vulnerability Database. https://nvd.nist.gov/vuln/detail/CVE-2017-5638

U.S. Government Accountability Office (2018). *Data Protection: Actions Taken by Equifax and Federal Agencies in Response to the 2017 Breach*, GAO-18-559. https://www.gao.gov/products/gao-18-559

U.S. House Committee on Oversight and Government Reform (2018). *The Equifax Data Breach*. https://oversight.house.gov/wp-content/uploads/2018/12/Equifax-Report.pdf

NIST (2022). *SP 800-40 Rev. 4, Guide to Enterprise Patch Management Planning*. https://csrc.nist.gov/pubs/sp/800/40/r4/final

Vulhub. *vulhub/struts2 image tags*, Docker Hub. https://hub.docker.com/r/vulhub/struts2/tags

ICDFA. *GRC102 Week 8 Lab Instructions*. https://github.com/icdfa/grc-engineering-labs/blob/master/phase1/grc102/week8/LAB_INSTRUCTIONS.md